

Ett webbaserat beslutsstöds- system för lönsamhetsanalys i last- bilsplanering

*A Web-Based Decision Support System for Profitability Analysis in
Truck Planning*

Jacob Ahling Hult
Simon Andersson
Elias Facq
Felix Hallström
Karl Lennartsson
Jakob Renqvist
Viktor Wallberg

Handledare : Szilvia Varro-Gyapay
Examinator : Lena Buffoni

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

Jacob Ahling Hult

Simon Andersson

Elias Facq

© Felix Hallström

Karl Lennartsson

Jakob Renqvist

Viktor Wallberg

Sammanfattning

Sammanfattningen finns i fil `Populärvetenskaplig sammanfattning` på svenska i `sammanfattning.tex`. Skriv din populärvetenskapliga sammanfattning här.

Abstract

The abstract resides in file `Abstract.tex`. Here you should write a short summary of your work. Write your abstract here.

Tillkännagivanden

Vi vill rikta ett varmt tack till Johan Munck, Jonas Lanfelt och Åke Axelsson på Börjes Koncernen för förtroendet att genomföra detta projekt samt för ett engagerat och tillmötesgående samarbete under hela projektets gång. Ett särskilt tack till vår handledare Szilvia Varro-Gyapay vid Linköpings universitet för vägledning, värdefulla synpunkter och kontinuerligt stöd under projektets samtliga faser. Slutligen vill vi tacka varandra i projektgruppen för ett engagerat och välorganiserat samarbete, det har gjort arbetet både lärorikt och roligt.

Innehåll

Sammanfattning	iii
Författarens tack	iv
Innehåll	v
Tabeller	vii
Definitioner	viii
1 Inledning	1
1.1 Motivering	1
1.2 Syfte	1
1.3 Frågeställningar	2
1.4 Avgränsningar	2
2 Bakgrund	3
2.1 Logistikbranschen	3
2.2 Börjes Koncernen	3
2.3 Projekt Visir	4
2.4 Projektgruppens bakgrund	4
3 Teori	6
3.1 Logistik och transportoptimering	6
3.1.1 Trafikledares beslutsstöd i logistikplanering	6
3.1.2 Prisestimering och lönsamhetsanalys	7
3.1.3 Vägens påverkan på kostnader och lönsamhet	8
3.1.4 Beslutsstödssystem i transportlogistik	8
3.2 Webbutveckling och systemarkitektur	8
3.2.1 Fullstack-arkitektur med Next.js	8
3.2.2 Frontend-layer: React-komponenter och användargränssnitt	9
3.2.3 Backend-layer: Next.js API-routes och serverlogik	9
3.2.4 Databaskonstruktion med SQL och Supabase	9
3.2.5 Datakommunikation mellan lager	9
3.2.6 iLog API-integration	10
3.2.7 Deployment och hosting	10
4 Metod	12
4.1 Förstudie och planering	12
4.1.1 Kravanalys	12
4.1.2 Tidsplanering och milstolpar	13
4.1.3 Dokumentation av kravspecifikation	13
4.2 Projektstruktur och arbetsmetoder	13

4.2.1	Möten och kommunikation	13
4.2.2	Versionshantering och Git	14
4.2.3	Kanban-arbetsflöde och Trello	14
4.2.4	Figma-design och prototyping	14
4.2.5	Roller och ansvar	15
4.3	Utveckling och systemarkitektur	15
4.3.1	Systemarkitektur och design	15
4.3.2	Frontend-utveckling: React-komponenter och användargränssnitt . . .	16
4.3.3	Backend-utveckling: Next.js API-routes och serverlogik	16
4.3.4	Databaskonstruktion: PostgreSQL och Supabase	17
4.3.5	iLog API-integration	17
4.3.6	Systemanatomi	18
4.4	Testning och kvalitetssäkring	18
4.4.1	Enhetstestning	18
4.4.2	Integrationstestning	18
4.4.3	System- och användbarhetstestning	18
4.4.4	Bugfix och iterativ förbättring	19
4.4.5	Hantering av bias vid testning	19
5	Resultat	20
5.1	Systembeskrivning	20
5.2	Processbeskrivning	20
5.3	Gemensamma erfarenheter	20
6	Diskussion	21
6.1	Resultat	21
6.2	Metod	21
6.3	Arbetet i ett vidare sammanhang	21
7	Slutsats	22
	Referenser	23

Tabeller

4.1	Översikt över roller och ansvarsområden inom projektgruppen	15
-----	---	----

Definitioner

- **Visir** - Systemets namn, kortare beteckning för "Projekt Visir".
- **iLog** - Börjes Koncernens befintliga logistik- och planeringssystem där trafikledare planerar lastbilar och bokningar.
- **Trafikledare** - Personal på Börjes Koncernen som är ansvarig för att planera lastbilar och bokningar i iLog.
- **Ekipage** - En lastbilskombination bestående av en dragbil och släp.
- **Flakmeter (flm)** - Måttenhet för lastkapacitet på en lastbil. En standard lastbil har kapacitet för 19,2 flakmeter.
- **Bokning/Frakt** - En enskild försändelse som ska transporteras, innehåller information om avsändare, mottagare, vikt, volym och önskat leveransdatum.
- **Lönsamhet** - I detta projekt definierat som den estimerade intäkten från en planerad lastbil. I senare faser kan lönsamhet även inkludera kostnader (intäkt minus kostnad).
- **Prisestimering** - Processen att uppskatta priset på en bokning som saknar redan satt pris, baserat på historisk data och algoritmer.
- **Taxepunktsrelation** – Kombinationen av avsändarens och mottagarens taxeområde (ungefär postnummerområde), används för att identifiera en specifik rutt i prisestimeringen. Skrivs som ett par.
- **Taxeområde** – Ett geografiskt område (ungefär postnummerområde), som utgör grunden för taxepunktsrelationer i Börjes Koncernens prissättning.
- **Fraktgrundande vikt** – Den vikt som ligger till grund för prissättningen av en bokning. Kan skilja sig från faktisk vikt om volymvikt är styrande.
- **Viktklass** – En klassificering av fraktgrundande vikt i fördefinierade intervall, används för att matcha bokningar mot historiska prisdata.
- **Traffiklinje** – Kombinationen av ett avsändarkontor och ett mottagarkontor, representerar en återkommande logistikförbindelse i Börjes Koncernens nätverk.
- **Trappstegsmodell** – Den estimeringsalgoritm som används i Projekt Visir för att uppskatta intäkten på bokningar utan fastställt pris. Modellen söker igenom sex steg med successivt lägre specificitet och stannar vid det första steget med tillräckliga historiska data.
- **Frontend** - Den del av systemet som körs i användarens webbläsare och ansvarar för det visuella gränssnittet.
- **Backend** - Den del av systemet som körs på servern och ansvarar för affärslogik, prisestimering och kommunikation med externa system.

- **API** - *Application Programming Interface*, ett gränssnitt som tillåter systemet att kommunicera med externa system för att hämta data.
- **REST-API** - Ett HTTP-baserat gränssnitt för att manipulera och hämta resurser med standardiserade metoder.
- **TypeScript** - Ett starkt typat programmeringsspråk byggt ovanpå JavaScript, används för både frontend och backend i Projekt Visir.
- **SQL** - *Structured Query Language*, ett språk för att hantera och hämta data ur relationsdatabaser.
- **Next.js** - Ett fullstack-ramverk byggt på React som hanterar både frontend och backend i samma applikation.
- **React** - Ett JavaScript-bibliotek för att bygga interaktiva användargränssnitt med komponentbaserad arkitektur.
- **Tailwind CSS** - Ett CSS-ramverk för att snabbt styla användargränssnitt direkt i komponentkod.
- **Node.js** - En körningsmiljö för JavaScript på serversidan, används av Next.js för backend-logik.
- **HTTP** - Hypertext Transfer Protocol. Standardiserat protokoll för överföring av data genom internet
- **Database Management System (DBMS)** - Ett mjukvarusystem som hanterar, lagrar och hämtar data på ett effektivt och strukturerat sätt från en databas. "Databashanterings-system" i svensk översättning.
- **Object-relational Database Management System (ORDBMS)** - Ett DBMS som också har stöd för objektorientering med objekt, klasser och arvshierarki.
- **PostgreSQL** - Ett ORDBMS-system med öppen källkod som används för att lagra transport- och lönsamhetsdata.
- **Supabase** - En molnbaserad databastjänst byggd på PostgreSQL som används för hosting och åtkomst till projektets databas.
- **Backend-as-a-Service (BaaS)** - System som gör det enkelt att bygga upp en databas eller *backend*. Supabase är ett BaaS-system.
- **State management** - Hantering av tillstånd i användargränssnittet, exempelvis valda filter, aktiv vy och hämtade resultat.
- **Read-only** - Endast läsbar åtkomst, vilket innebär att data kan hämtas men inte ändras.
- **Caching** - Mellanlagring av data för att minska svarstider och undvika upprepade hämtningar.
- **LaTeX** - Ett typsättningssystem som används för att utforma vetenskapliga och tekniska dokument med hög typografisk kvalitet.
- **Overleaf** - En webbaserad editor för LaTeX-dokument som möjliggör samarbete i realtid.
- **GitHub** - En plattform för versionshantering och samarbete kring kod, baserad på Git.
- **Figma** - Ett webbaserat design- och prototyperingsverktyg som användes för att ta fram systemets användargränssnitt.

- **Kanban** - En arbetsmetod för att visualisera och hantera arbetsflöde, implementerad genom det digitala verktyget Trello.
- **Trello** - Ett digitalt Kanban-verktyg som användes för att fördela och följa upp arbetsuppgifter under projektet.



1 Inledning

Detta kapitel presenterar introduktionen för Projekt Visir. Vi introducerar läsaren till de problem som motiverat projektet, definierar projektets syfte, frågeställningar och redogör för projektets avgränsningar.

1.1 Motivering

Inom transportlogistiken står Börjes Koncernen inför en stor utmaning, hur säkerställer man att de lastbilar som planeras verkligen är lönsamma innan de genomförs? I dag förlitar sig trafikledare primärt på erfarenhet och intuition när de planerar vilka frakter som ska kombineras på en lastbil. Detta utgör ett verkligt problem eftersom lönsamhetsdata saknas direkt i planeringsgränssnittet.

Resultat av denna informationsbrist är ofta suboptimala planeringsbeslut. Trafikledare kan missa möjligheter att justera sammansättningar i realtid, eller ägnar de tid på att manuellt samla information från olika system för att bedöma om en planering är rimlig ekonomiskt. I en bransch där marginalerna är täta och konkurrensen är skarp, utgör detta en konkurrenssnackdel.

Projekt Visir är utvecklat för att lösa detta problem genom att integrera prisestimering och lönsamhetskalkyl direkt i trafikledarens arbetsflöde. Systemet hämtar planeringsdata från planeringsgränssnittet via API, estimerar intäkter baserat på historisk data, och presenterar ett visuellt verktyg som gör det möjligt för trafikledare att direkt bedöma om de ska justera planeringen. Detta är ett intressant systemutvecklingsprojekt eftersom det kräver integration mot ett befintligt komplext system, hantering av stora datamängder, implementering av algoritmer för prisestimering, säker datakommunikation, och utformning av ett användargränssnitt optimerat för snabba beslut.

1.2 Syfte

Denna kandidatrapport presenterar Projekt Visir ur tre huvudsakliga perspektiv. Det första perspektivet är systembeskrivningen, där vi redovisar hur systemet är utformat, vilka komponenter som implementerades, vilka val som gjordes gällande teknologi och design, och

hur systemet möter de krav som ställdes. Det andra perspektivet är processbeskrivningen, där vi dokumenterar hur gruppen organiserade sitt arbete, vilka utvecklingsmetoder som tillämpades, hur vi mätte processens kvalitet, och vilka förbättringar vi genomförde under projektets gång. Det tredje perspektivet är erfarenhetssamlingen, där vi reflekterar över det vi lärt oss och vilket värde dessa insikter kan ha för framtida projekt.

Rapporten syftar således till att ge läsaren en helhetsbild av projektet, vad vi byggde, hur vi byggde det, och vad vi lärt oss. För examinator och handledare ger den underlag för att bedöma både produkten och processen. För Börjes Koncernen kan den fungera som referensdokument för hur systemet är konstruerat och vilka beslut som låg till grund för utformningen samt för hur de ska kunna vidarearbeta på projektet.

1.3 Frågeställningar

Denna rapport adresserar följande fyra forskningsfrågor. De tre första är givna från kurskraven:

1. Hur kan projekt Visir implementeras så att man skapar värde för Börjes Koncernen?
2. Vilka erfarenheter kan dokumenteras från programvaruprojekt Visir som kan vara intressanta för framtida projekt?
3. Vilket stöd kan man få genom att skapa och följa upp en systemanatom?

Den fjärde frågan är projektspecifik:

4. Hur kan en trappstegsbaserad prisestimering med historisk data öka trafikledarnas förmåga att planera lönsamt och fatta bättre beslut i realtid?

1.4 Avgränsningar

Projektet har följande avgränsningar:

1. För det första fokuserar systemet på lönsamhetsanalys av redan planerade lastbilssamlingar i iLog och optimerar inte själva planeringsprocessen.
2. För det andra är systemet utformat som en read-only integration mot iLog, vilket innebär att ingen data skrivs tillbaka till iLog-systemet. Detta är en medveten design för att säkerställa dataintegritet i kundens befintliga operativa system.
3. För det tredje täcker systemet inte faktisk körning eller leverans, utan fokuserar helt på planeringsfasen.
4. För det fjärde är implementeringen av prisestimerings-algoritmen begränsad till de data och metodologier som Börjes Koncernen tillhandahåller. Projektgruppen implementerar inte nya algoritmer utan använder befintliga logiker enligt kundens specifikation.
5. Slutligen är denna rapport fokuserad på implementation av version 1.0 av systemet. Framtida utbyggnader och förbättringar av systemet ligger utanför denna rapports omfattning.



2 Bakgrund

Detta kapitel presenterar den organisatoriska, marknadsrelaterade och projektspecifika kontexten för Projekt Visir. Vi beskriver Börjes Koncernen som organisation, undersöker logistikbranschen, motiveringen för projektet och presenterar gruppens bakgrund och erfarenheter.

2.1 Logistikbranschen

Transportlogistiken utgör en kritisk del av den moderna ekonomin och är en av Sveriges största näringar. Den svenska logistik- och lagerbranschen nådde ett värde på 185 miljarder SEK 2023, driven av landets strategiska geografiska läge, stark tillverknings- och exportsektor, samt växande efterfrågan på leveranstjänster sista milen inom e-handeln [14]. Branschen står emellertid inför betydande utmaningar som påverkar lönsamheten. De operationella kostnaderna för logistikföretag ökade med nästan 11 procent under 2023 på grund av stigande bränsle-, el- och personalkostnader, en trend som särskilt drabbar små och medelstora aktörer. Arbetskraftsbrist utgör också ett växande problem, nästan 25 procent av logistikföretagen rapporterade detta som sin största utmaning 2023 [14]. Infrastrukturflaskhalsar i tätortsområden, där leveransförseningar ökat med 18 procent under de senaste två åren, skapar ytterligare osäkerhet.

E-handelns framväxt öppnar samtidigt nya möjligheter. E-handeln i Sverige växte med över 14 procent under 2023, vilket driver efterfrågan på snabbare och mer flexibla leveranslösningar [14]. För att möta denna efterfrågan investerar företag i effektivare planeringsverktyg och beslutsstödsystem. Digitalisering och automation börjar redan transformera branschen, många större aktörer har implementerat realtids-spårning, automatiserade lager och AI-drivna system [2]. Dock har många medelstora och mindre företag ännu inte fullt ut utnyttjat möjligheterna med datadrivna verktyg för planering och optimering, vilket skapar en möjlighet för företag som kan implementera smarta lösningar för att vinna konkurrensfördelar genom bättre planeringsunderlag och ökad lönsamhet per körning.

2.2 Börjes Koncernen

Börjes Koncernen är en etablerad aktör inom svensk transportlogistik med över 70 års historia sedan grundandet 1947 [1]. Koncernen är idag en betydande logistikleverantör som syssel-

sätter cirka 600 medarbetare och förfogar över mer än 300 lastbilar och släp. Huvudkontoret ligger i Nybro och koncernen äger flera operativa bolag, däribland Smålands Logistik, som är koncernens största och mest expanderande enhet. Genom dessa bolag erbjuder Börjes Koncernen ett omfattande utbud av logistiklösningar inklusive distributionstrafik, fjärrtrafik, kyl- och frystransporter, samt tredjepartslogistik [1]. Koncernen har också etablerat sig som en innovatör inom hållbar logistik, med investeringar i eldrivna fordon och högkapacitets-ekipage för att minska miljöpåverkan.

Börjes Koncernen står inför ett föränderligt marknadslandskap där konkurrensen intensifieras och kundernas krav på effektivitet och transparens ökar. E-handelns expansion och digitalisering av logistikbranschen driver denna förändring, samtidigt som operationella kostnader stiger [14]. För att behålla sin konkurrensposition insåg koncernen behovet av att modernisera sina planeringsprocesser och implementera datadrivna verktyg som kan öka lönsamheten per körning och ge trafikledare bättre planeringsunderlag. Detta utgör bakgrunden till Projekt Visir, som utvecklats för att ge trafikledare vid Börjes Koncernen tillgång till realtidsanalys av lastbilsplaneringarnas lönsamhet, och därmed möjliggöra snabbare och bättre beslut i den dagliga trafikplaneringen.

2.3 Projekt Visir

Projekt Visir är utvecklat för att lösa en viktig utmaning i trafikledarnas dagliga arbete, möjligheten att snabbt och tillförlitligt bedöma lönsamheten hos en planerad lastbil innan den godkänns. Som beskrivits tidigare fattas trafikledare idag ofta dessa beslut utan tillgång till prisdata eller historiska jämförelser, vilket resulterar i suboptimala planeringsbeslut och missad lönsamhet.

Visir är ett webbaserat system som integrerar med Börjes Koncernens befintliga planeringssystem för att ge trafikledare realtidsanalys av lastbilsplaneringarnas ekonomiska status. Systemet använder historisk transportdata för att estimerar intäkter på bokningar som saknar redan fastställt pris, och presenterar detta visuellt så att trafikledare snabbt kan bedöma om en sammansättning är lönsam eller behöver justeras. Systemet erbjuder också simuleringsmöjligheter, som gör det möjligt för trafikledare att testa olika planeringalternativ innan godkännande, en funktion som möjliggör iterativ förbättring av planeringen utan att behöva lämna systemet.

Genom att integrera prisestimering, lönsamhetsanalys och simulering i ett enda gränssnitt syftar Projekt Visir till att ge trafikledare ett starkt verktyg för att fatta bättre planeringsbeslut i realtid. Detta förväntas leda till högre intäkter per körning, bättre resursutnyttjande, och ökad konkurrenskraft för Börjes Koncernen. Projektet representerar också en betydande systemutvecklingssatsning, det kräver integration mot ett befintligt komplext system, hantering av stora datamängder, och utformning av ett användargränssnitt optimerat för snabba beslut under tidstryck.

2.4 Projektgruppens bakgrund

Projektgruppen består av sex datateknologer och en mjukvaruteknolog som alla läser kursen TDDD96 - Kandidatprojekt i programvaruutveckling vid Linköpings Universitet. Det huvudsakliga kursmomentet är att utveckla en produkt åt en utomstående beställare, vilket utgör en ny upplevelse för alla i projektgruppen. Samtliga projektmedlemmar har tidigare erfarenhet av att arbeta i projekt av mer informell karaktär, där de har haft en beställare i form av kursansvarig men där projekten inte varit förankrade i ett verkligt företagsbehov till samma grad som Projekt Visir.

Denna övergång från akademiska projektuppgifter till ett verkligt utvecklingsprojekt åt en etablerad logistikoperatör har påverkat gruppens angreppssätt. Gruppen har satsat på tydlig struktur, regelbundna möten och transparent kommunikation för att säkerställa att krav förstås korrekt och att projektmål nås. Det faktum att detta är gruppens första gemensamma projekt har också resulterat i att medlemmar snabbt etablerat roller baserat på individuella styrkor och tidigare erfarenheter, vilket möjliggjort en effektiv arbetsfördelning från projektets start.



3 Teori

Syftet med detta kapitel är att presentera den teoretiska bakgrunden för både logistik-delen och den tekniska implementationen av Projekt Visir. Vi beskriver relevanta teorier från transportlogistik och presenterar de tekniska ramverk som användes för systemutveckling.

3.1 Logistik och transportoptimering

Transportlogistik handlar i grunden om att flytta gods från punkt A till punkt B på ett sätt som är ekonomiskt hållbart för alla inblandade parter. För att detta ska fungera så krävs dels en grundläggande förståelse för vad en transport faktiskt kostar, dels de verktyg som gör det möjligt att fatta välgrundade beslut under tidspress. Projekt Visir befinner sig mellan dessa behov. Systemet syftar till att ge trafikledare tillgång till lönsamhetsdata direkt i planeringsgränssnittet, vilket förutsätter en teoretisk grund i både prismodellering och beslutsstödst teori. Detta delkapitel presenterar den logistiska teorin bakom Projekt Visir, med fokus på trafikledares behov, principer för prisestimering och lönsamhetsanalys, hur transportmiljöns fysiska egenskaper påverkar kostnader, samt vad som utmärker ett effektivt beslutsstödsystem.

3.1.1 Trafikledares beslutsstöd i logistikplanering

I transportlogistikens dagliga verksamhet är trafikledaren ansvarig för att sammansätta lastbilar, det vill säga att bestämma vilka bokningar som kombineras på samma körning, vid vilken tidpunkt och med vilket fordon. Dessa beslut fattas i ett högt tempo och med begränsad tid per planering. En viktig utmaning är att trafikledaren sällan har omedelbar tillgång till ekonomisk information om den planering som håller på att skapas. Besluten grundas i stället primärt på erfarenhet, känsla för rutten och relationer till kunder snarare än den faktiska lönsamheten.

Denna informationsbrist leder till ett välkänt problem inom logistikplaneringen, suboptimala beslut som inte uppstår i brist på kompetens utan brist på beslutsunderlag. Utan tillgång till realtidsdata om intäkter och kostnader är det svårt att avgöra om en sammansatt lastbil är lönsam, om den bör justeras eller om vissa bokningar bör omfördelas. Trafikledaren tvingas antingen fatta beslut på osäker grund eller lägga tid på att manuellt samla informa-

tion från flera olika system, vilket i sig är tidskrävande eller feltänkt.

Forskning visar att logistikplanerare som saknar bra systemstöd ofta måste väga lönsamma och olönsamma körningar mot varandra för att få en acceptabel genomsnittlig vinst [8]. Det gör att varje enskild körning inte alltid blir så bra som möjligt. Resultatet är att man går miste om en potentiell vinst och att resurserna inte används på bästa sätt. Med ett system som visar lönsamheten för varje körning direkt kan planeraren fatta bättre beslut och arbeta mer effektivt med logistiken.

3.1.2 Prisestimering och lönsamhetsanalys

Grunden för lönsamhetsanalys i transportlogistik är förmågan att estimerar intäkten för en enskild körning. I praktiken är det inte så enkelt eftersom inte alla bokningar har ett fastställt pris vid planeringstillfället. Prisestimering avser processen att uppskatta en förväntad intäkt baserat på tillgänglig information och historisk data.

Det dominerade sättet att prissätta lastbilstransporter inom branschen är att basera ersättningen på körd sträcka med lastbil. En vanlig modell uttrycks som en linjär funktion $q(d) = a_1 + a_2 \cdot d$, där $q(d)$ representerar priset per enhet gods vid transportsträckan d och a_1 respektive a_2 är förhandlade parametrar [8]. Modellen ger en god approximation av genomsnittskostnaden över ett stort antal transporter men kan vara missvisande för enskilda körningar, eftersom den inte fångar variation i transportmiljön eller kundspecifika avtal [3].

Inom Projekt Visir används en trappstegsbaserad estimeringsmodell som utnyttjar historiska transportdata för att uppskatta intäkten på bokningar utan fastställt pris. Modellen söker efter jämförbara historiska transporter genom en sekvens av steg med successivt lägre specificitet och stannar vid det första steget där tillräcklig historisk data finns.

Modellens sex steg tillämpar följande sökkriterier, i fallande ordning av specificitet.

1. **Kundnamn + taxepunktsrelation + viktklass** - Det mest specifika steget matchar på kombination av kundnamn, taxepunktsrelation (det vill säga avsändarens och mottagarens taxeområde, i praktiken ett par av postnummer) samt viktklassen för den fraktgrundande vikten. En träff här innebär att exakt samma kund tidigare har skickat gods av liknande vikt på samma rutt.
2. **Kundnamn + viktklass** - Matchar på kundnamn och viktklass oavsett rutt. Används när kundspecifik historisk data finns men inte för den specifika taxepunktsrelationen.
3. **Kundnamn** - Matchar enbart på kundnamn. Använder ett genomsnitt av kundens historiska prisnivå, skalat med en förhållandekoefficient mot det generella svenska snittet.
4. **Taxepunktsrelation + viktklass + avstånd** - Steget lämnar kundspecifika data och matchar i stället på taxepunktsrelation, viktklass och avståndsintervall. Representativt pris hämtas ur en tabell över genomsnittliga priser för körningar med liknande rutt och vikt.
5. **Linje + viktklass + avstånd** - Matchar på trafiklinjen (kombinationen av avsändare och mottagare), viktklassen och avståndet. Linjetabellen ger ett genomsnittspris för körningar på samma logistiklinje.
6. **Avstånd + viktklass** - Det minst specifika steget och den sista utvägen. Estimeringen baseras uteslutande på det körda avståndet och viktklassen, utan hänsyn till kund eller rutt.

Lönsamhet per körning definieras i Projekt Visir som den estimerade totala intäkten från alla bokningar på en planerad lastbil. Detta ger trafikledaren ett jämförbart mått per körning och mot historiska referensvärden.

3.1.3 Vägens påverkan på kostnader och lönsamhet

Även om Visirs estimeringsmodell bygger på historisk data är det viktigt att förstå en grundläggande begränsning i distansbaserade prismodeller. Transportmiljöns fysiska egenskaper påverkar den faktiska kostnaden markant, oberoende av körningens längd [3, 8]. Faktorer som backighet, vägkvalitet, kurvighet, hastighetsgränser och korsningar driver bränslekonsumtion och körtid på sätt som en sträckbaserad formel inte fångar. Rönnqvist et al. [8] illustrerar detta konkret: tre körningar med identisk sträcka och identisk distansbaserad ersättning kan ha väsentligt olika faktiska kostnader beroende på om rutten går på motorväg, landsvängsmix eller smal skogsväg med många korsningar. I ett nordiskt sammanhang tillkommer dessutom vinterspecifika utmaningar i kuperad terräng, vilket påverkar körtider och säkerhet [3].

Dessa insikter är relevanta för Projekt Visir som tolkningsperspektiv. Körningar i ovanligt svåra eller ovanligt enkla transportmiljöer kan ge en estimering som avviker från det faktiska utfallet, även om historisk data i övrigt är representativt. Att vara medveten om denna begränsning är viktigt vid tolkning av systemets utdata och motiverar framtida förbättringar där transportmiljöns karaktäristik vägs in i estimeringsmodellen.

3.1.4 Beslutsstödssystem i transportlogistik

Ett beslutsstödssystem *Decision Support System (DSS)* är ett informationssystem utformat för att stödja beslutsfattande i komplexa situationer där data, analys och mänskligt omdöme behöver förenas [8]. Inom transportlogistik avser detta typiskt system som hjälper planerare att fatta välgrundade beslut om rutter, lastbilssammansättningar och prissättning utan att ersätta planerarens kompetens och erfarenhet.

För att vara användbart i realtidsplanering behöver ett DSS kombinera tre egenskaper. Det måste erbjuda realtidsinformation direkt i gränssnittet, ge visuellt lättolkad återkoppling som kan uppfattas under tidspress och erbjuda simuleringsmöjligheter som låter planeraren testa alternativ innan beslut fattas. Dessa egenskaper transformerar systemet från ett passivt rapporteringsverktyg till ett aktivt planeringsstöd. Projekt Visir är utformat med dessa krav som utgångspunkt och kopplar samman den trappstegsbaserade prisestimering med ett gränssnitt som är optimerat för snabba beslut i trafikledarens dagliga arbete.

3.2 Webbutveckling och systemarkitektur

Detta delkapitel presenterar det tekniska ramverk och arkitektur för Projekt Visir, som är byggt som en integrerad Fullstack-applikation med Next.js. Vi förklarar hur frontend- och backend-komponenter är sammanflätade i samma ramverk och hur de kommunicerar med databaser och externa API:er.

3.2.1 Fullstack-arkitektur med Next.js

Projekt Visir är en Fullstack-applikation, vilket betyder att frontend (React-komponenter som körs i webbläsare) och backend (serverlogik som körs på servern) är integrerade i samma Next.js-ramverk och driftsätts tillsammans på Vercel. Detta är annorlunda än traditionell arkitektur där frontend och backend är helt separata. Med Next.js får utvecklare båda miljöerna i samma projekt, vilket förenklar utveckling och driftsättning.

3.2.2 Frontend-layer: React-komponenter och användargränssnitt

React.js är ett JavaScript-bibliotek för att hjälpa utvecklare skapa mer strukturerade, interaktiva och snyggare användargränssnitt [7]. React.js bygger på ett komponentbaserat arbetssätt där varje del av gränssnittet - exempelvis knappar, navigeringsfältet eller hela sidsektioner - skapas av utvecklaren som återanvändbara komponenter. En komponent har samma funktion som en JavaScript funktion, men fungerar genom att isolera ett kodstycke och sedan returnera HTML. Systemet använder sig av *state management* vilket innebär att klienten sparar serverdata som en form av cache. Detta för att synkronisera data mellan klient och server utan att behöva skicka onödigt mycket data. För att hantera navigation mellan dem olika sidorna så använder systemet *React Router*. Det är ett kraftfullt bibliotek som gjort systemet mer responsivt. Tailwind CSS bygger på CSS, som namnet antyder, vilket är ett *open-source framework* av en kollektion med färdiga UI element.

3.2.3 Backend-layer: Next.js API-routes och serverlogik

Ett Next.js-projekt har för närvarande två metoder att definiera *backend*-delen av projektet, *App router* och *Pages router* [16, 17]. Båda dessa tillvägagångssätt har som konvention att exponera en API-rutt per `route.ts` fil i varje undermapp till `app/api` eller `pages/api` beroende på metod. Huvudskillanden mellan de två metoderna är att *App router* kan hantera olika HTTP-metoder i olika exponerbara funktioner medan *Pages router* enbart kan exponera en funktion för samtliga HTTP-metoder. Följaktligen är också *App router* modernare medan *Pages router* är ett äldre system. I projekt visir används metoden *App router*.

3.2.4 Databaskonstruktion med SQL och Supabase

Supabase är ett kraftfullt *Backend-as-a-Service*-system som bidrar med en bred nomenklatur av moderna funktioner, exempelvis ett automatiskt genererat REST-API inbyggt i Next.js, modern kryptering av känslig data, inbyggt autentiseringssystem med *JSON Web Tokens* (JWT) samt arbiträrt definierbara skydd till data i form av deras *Row Level Security*-system [13]. I centrum av Supabase är en relationsdatabas som bygger på PostgreSQL, som i sin tur bygger på det minimala SQL-språket.

PostgreSQL är ett *Object-relational database management system* (ORDBMS) som är baserat på POSTGRES utvecklat av Berkley University mellan 1986 och 1996 [6]. År 1996 gick systemet igenom en stor omarbetning och döptes om till PostgreSQL, nu med fokus på att utveckla nya funktioner som *foreign keys*, *triggers*, och *aggregate functions*. PostgreSQL är idag ett mycket effektivt och snabbt DBMS i jämförelse med dess alternativ. Systemet är upp till 13 gånger snabbare än det alternativa databashanteringssystemet MySQL [10] vilket är en av anledningarna till att Supabase använder PostgreSQL.

3.2.5 Datakommunikation mellan lager

Next.js och Supabase är mycket samarbetande system. Kommunikation mellan dessa två system sker på så sätt väldigt smidigt. Med hjälp av det automatiskt genererade API:et i Supabase, som introducerades i kapitel 3.2.4, kan Next.js programmatiskt skapa enkla operationer som liknar rena SQL-satser. Detta ger upphov till ett enkelt och välkänt sätt för hur *backend* i Next.js och databasen i Supabase kan kommunicera.

Medan detta API automatiskt genereras på Supabase-sidan av projektet så har Next.js inte automatisk uppsikt om databasens struktur. Detta löses genom att automatiskt generera en TypeScript-fil som lagrar databasens struktur genom kommandoprompten för Supabase. På så sätt vet Next.js hur strukturen på databasen ser ut, vilket gör att operationer på

tabeller med arbiträra namn kan kontrolleras av kompilatorn och behöver inte vänta tills exekveringen sker.

3.2.6 iLog API-integration

iLog tillhandahåller ett REST-baserat webb-API som möjliggör för externa system att hämta och modifiera planeringsdata. Kommunikation sker över HTTP och svar returneras i JSON-format med en struktur som indikerar om anropet lyckades samt innehåller efterfrågande data.

iLog-API:et stödjer både läs- och skrivoperationer. För Projekt Visir används endast GET-anrop, det vill säga läsoperationer och inga skrivoperationer (POST) genomförs mot systemet. Detta är ett medvetet designval som grundar sig i att iLog är ett produktionssystem i aktiv drift hos Börjes Koncernen och många andra aktörer. Genom att avstå från skrivoperationer minimeras risken för att påverka pågående verksamhet vid felaktig användning.

Den data som är relevant och hämtas är huvudsakligen tre informationsmängder. Det första är hämtning av översiktsdata över tillgängliga resurser i form av ekipage kopplade till en viss aktör. Det andra är hämtning av planeringsdata i form av alla bokningar kopplade till ett specifikt ekipage under ett givet tidsintervall. Det tredje är hämtning av detaljerad information om en enskild bokning, exempelvis uppgifter relaterande till bokningens innehåll och omfattning.

Eftersom iLog är ett produktionssystem som används av ett stort antal aktörer samtidigt är det viktigt att minimera antalet API-anrop för att inte belasta systemet. Av denna anledning är det tänkt att Projekt Visirs datahämtningsstrategi är utformad för att köra ett begränsat antal anrop, avsett att ske automatiskt under perioder med låg systembelastning hos aktörerna. Data från iLog chachelagras sedan i projektets databas i Supabase, varifrån den görs tillgänglig för frontend under arbetstid utan att ytterligare API-anrop mot iLog behöver göras. Detta innebär att trafikledare under en arbetsdag arbetar mot en lokal kopia av föregående dags planeringsdata, vilket ger snabba svarstider i gränssnittet samtidigt som det skyddar iLog-systemet från onödig belastning.

3.2.7 Deployment och hosting

Projekt Visir driftsätts som en sammanhållen Next.js-applikation på Vercel, en molnbaserad plattform optimerad för just Next.js [18]. Eftersom Next.js integrerar frontend och backend i samma projekt hanterar Vercel båda lagren inom samma miljö. Frontend-komponenterna levereras som statiskt eller serverrenderat innehåll, medan backend-logiken i form av API-routes körs som serverlösa funktioner [19]. Dessa utgör isolerade funktionsinstanser som startas vid behov och skalas automatiskt med inkommande trafik.

Vercel hanterar även HTTPS-konfiguration, global distribution av statiska resurser via ett CDN (filer lagras på serverar runt om i världen, användare får snabbare laddning) samt automatisk driftsättning kopplad till projektets GitHub-repository [15]. Detta innebär att varje uppdatering av produktionsbranchen automatisk bygger och publicerar en ny version av applikationen.

Databasen hanteras av Supabase, en molntjänst byggd ovanpå PostgreSQL [12]. Utöver databashanteringen erbjuder Supabase även autentisering och ett REST-API som genereras direkt från databasschemat [11]. I Projekt Visir används Supabase främst som bestående lagring för cachelagrad iLog-data, för de prisestimering- och lönsamhetsdata som backend beräknar och för den historiska datan som importeras från KUSK.

Kombinationen av Vercel och Supabase innebär att Projekt Visir inte kräver någon egen serverinfrastruktur att underhålla. Skalning sker automatiskt i båda lagren och tjänsterna erbjuder inbyggd redundans samt hög tillgänglighet utan manuell konfiguration. Denna arkitektur är väl lämpad för ett projekt av Projekt Visirs storlek och livscykel, där enkel driftsättning och lågt underhåll prioriteras framför möjligheten att finjustera den underliggande infrastrukturen.



4 Metod

Syftet med detta kapitel är att dokumentera hur projektet genomfördes från första steg genom förstudie, tidsplanering, kravanalys och planering genom design, utveckling, testning och leverans. Vi beskriver de metoder som användes, projektets organisering och de steg som följdes för att bygga Projekt Visir enligt kundens krav.

4.1 Förstudie och planering

Projektet inleddes med en förstudie för att skapa en förståelse för uppdraget samt de krav och behov som identifierats av kunden. Denna process bidrar till att undvika ogenomtänkta snabba beslut, stress i senare faser, osäkerhet i resultatet samt ett ej målinriktat projektarbete [20].

Projektet bestod av att utveckla en webbaserad lösning där en trafikledare i realtid kan ange transportinformation och utifrån tidigare data se lönsamheten. Som en del av förstudien deltog vi i ett flertal seminarier inom kursen TDDD96. Under dessa seminarier diskuterades projektplanering, gruppstruktur och olika arbetssätt och praxis. Seminarierna bidrog till en ökad förståelse för hur projektet kunde organiseras och hur gruppen skulle kunna arbeta effektivt.

4.1.1 Kravanalys

För att identifiera projektets funktionella och icke-funktionella krav genomfördes regelbundna möten med kunden. Under dessa möten presenterades kunden sin vision för produkten samt hur de föreställde sig att systemet skulle användas i praktiken. Detta gav en bättre förståelse för verksamhetens behov och vilka funktioner systemet behövde stödja.

Utifrån informationen från mötena formulerades ett antal preliminära funktionella och icke-funktionella krav. Dessa sammanställdes och presenterades för kunden för granskning och återkoppling. Kunden fick därmed möjlighet att kommentera och revidera kraven för att säkerställa att de överensstämde med företagets behov och förväntningar. Efter återkoppling fördes en fortsatt dialog för att ytterligare tydliggöra produktens funktionalitet och använd-

ningsområde. Denna iterativa process bidrog till att förtydliga projektets kravbild och låg till grund för kravspecifikationen.

4.1.2 Tidsplanering och milstolpar

För att strukturera projektets arbete skapades en tidsplan som utgick från både kundens förväntningar och de deadlines som fastställts inom kursen TDDD96. Dessa riktlinjer användes för att dela upp projektet i ett antal övergripande faser, vilket bidrog till att skapa en tydlig arbetsstruktur under projektets gång.

Projektets organiserades i flera faser, däribland förstudie, kravanalys, systemdesign, implementation, testning samt rapportskrivning. Genom denna uppdelning kunde arbetet planeras och genomföras stegvis, samtidigt som framsteg kontinuerligt kunde följas upp. Detta underlättade att identifiera eventuella behov av justering.

Som en del av tidsplaneringen definierades även ett antal milstolpar. Dessa milstolpar användes för att följa upp projektets utveckling och säkerställa att arbetet fortsatte enligt plan. Milstolparna inkluderade bland annat godkänd projektplan och kravspecifikation, färdig systemarkitektur, implementerad databasåtkomst via API och fungerande prototyp.

4.1.3 Dokumentation av kravspecifikation

Som ett resultat av kravanalysen sammanställdes en kravspecifikation där projektets identifierade krav dokumenterades. Syftet med dokumentet var att skapa en tydlig och gemensam referens för alla involverade parter.

Kravspecifikationen innehöll både funktionella och icke-funktionella krav som definierades utifrån de möten och diskussioner som förts med kunden. De funktionella kraven beskrev systemets centrala funktioner, exempelvis *systemet ska validera att båda fälten är ifyllda innan beräkning startas*. De icke-funktionella kraven behandlade bland annat aspekter som användbarhet och prestation.

Dokumentet används som ett stöd vid design, implementation och testning av systemet. Genom att utgå från kravspecifikationen kunde det säkerställas att utvecklingen av systemet överensstämde med de krav och förväntningar som identifierats tillsammans med kunden. Vid behov kan dokumentet uppdateras för att återspegla eventuella förtydliganden eller ändringar som uppstår under projektets gång.

4.2 Projektstruktur och arbetsmetoder

Detta delkapitel presenterar hur projektet organiserades och hur teamet arbetade tillsammans. Vi beskriver de möten som genomfördes, kommunikationskanaler som användes, och Kanban-metoden som strukturerade dagligt arbete.

4.2.1 Möten och kommunikation

För att säkerställa kontinuerlig samordning både inom projektgruppen och i relation till externa intressenter hade gruppen flera typer av mötesformer. En central del var de veckovisa handledarmötena. Inför dessa möten upprättade teamledaren en agenda, vilket möjliggjorde att projektgruppen och handledaren kunde förbereda sig. Under mötet presenterades projektets status, inklusive genomförda aktiviteter, pågående arbete samt planering av nästa steg. Vidare gavs gruppmedlemmarna möjlighet att bidra med synpunkter och reflektioner. Syftet med dessa möten var att säkerställa projektets utveckling fortsatt i enlighet med uppsatta

mål, samt skapa utrymme för att diskutera frågor och utmaningar.

Utöver handledarmöten genomfördes regelbundna interna möten inom projektgruppen. Dessa lade fokus på tekniska lösningar, arbetsfördelning och funderingar. I samband med dessa möten producerade vi även projektets statusrapport och tidsrapport. De interna mötena präglades av en hög grad av flexibilitet och anpassades efter aktuella behov, vilket innebär att deltagandet kunde variera beroende på arbetsområde och relevans för gruppmedlemmar.

Projektgruppen hade även möten med beställaren, vilka planerades och leddes av den analysansvarige. Dessa möten har främst syftat till att tydliggöra och fördjupa sig i beställarens krav och förväntningar. Samtliga möten med beställaren har genomfördes på distans via Microsoft Teams. Vår beställare bestod av flera olika specialister, vilket medförde en djupare förståelse av olika områden.

För att kommunicera snabbt och informellt inom gruppen användes Messenger. Denna plattform används för daglig samordning och kortare frågor. För mer utförlig kommunikation, delning av dokument, bilder och genomföring av vissa interna möten används Discord. För kommunikation med handledaren använde vi oss av Microsoft Teams, där vi även publicerade mötesagendor och projektets dokumentation delades.

4.2.2 Versionshantering och Git

Versionshantering genomfördes med hjälp av GitHub, vilket möjliggjorde en strukturerad utveckling samt säkerställde möjligheten till vidareutveckling av projektet i framtiden. I arbetet användes en grenstruktur där olika delar av systemet utvecklades i separata branches. Exempelvis användes en separat branch för frontend-utveckling samt en för produktionsversionen av webbapplikationen.

För att minska risken för konflikter organiserades utvecklingsarbetet så att olika funktionella områden hanterades parallellt. Sammanfogning av kod skedde genom etablerade versionshanteringsrutiner, vilket bidrog till en mer kontrollerad och överskådlig utvecklingsprocess.

4.2.3 Kanban-arbetsflöde och Trello

Under projektets genomförande tillämpades Kanban-metoden som arbetsflöde. Metoden innebar att arbetsuppgifter strukturerades i olika faser, "To do", "Doing" och "Done", vilket lade grunden för en kontinuerlig överblick av projektets status. Forskning visar att Kanban-metoden förbättrar arbetsprocessen samt tydliggör arbetsuppgifter. [5]

För användningen av metoden användes Trello, ett verktyg som visualiserar hanteringen av uppgifter. I Trello representerades arbetsuppgifter som kort, vilket organiserades utifrån arbetsflödets olika faser. Korten tilldelades utvecklingsområde, såsom frontend, arkitektur, databas osv., och kunde även tilldelas specifika gruppmedlemmar. Detta bidrog till en ökad transparens i arbetsfördelningen. Arbetsflödet möjliggjorde även en efterföljande utvärdering av arbetes genomförande.

4.2.4 Figma-design och prototyping

Under projektets tidigare faser för att etablera en gemensam förståelse för systemets design i samråd med beställaren användes Figma, ett webbaserat verktyg för design och prototypframtagning.

Designarbetet genomfördes iterativt, där prototyper successivt utvecklades och reviderades baserat på återkoppling från beställaren. Denna process bidrog till att säkerställa att designen överensstämde med beställarens krav och förväntningar. När en tillfredsställande prototyp hade etablerats fungerade denna som grund för den efterföljande implementationen.

4.2.5 Roller och ansvar

För att strukturera arbetet inom projektgruppen definierades ett antal roller med specifika ansvarsområden, se tabell 4.1. Dessa roller syftade till att tydliggöra ansvarsfördelningen samt säkerställa en effektiv arbetsprocess. Utöver dessa roller fördelades det praktiska utvecklingsarbetet baserat på funktionella ansvarsområden. Projektgruppen delades huvudsakligen in i frontend- och backend-utveckling. Frontend-utvecklingen ansvarade för att utforma och implementera användargränssnittet samt visualisera data, medan backend-utvecklingen ansvarade för systemets bakomliggande logik, inklusive datahantering, databaskopplingar och arkitektur.

Tabell 4.1: Översikt över roller och ansvarsområden inom projektgruppen

Roll	Ansvar
Teamledare	Hade det övergripande ansvaret för projektets framdrift samt för att förmedla information från handledare och examinator till projektgruppen. Rollen innefattade även att representera gruppen utåt samt att fatta övergripande beslut.
Analysansvarig	Ansvarade för den primära kontakten med beställaren samt för att identifiera, tolka och förmedla krav till projektgruppen.
Utvecklingsledare	Ansvarade för att leda utvecklingsarbetet, fatta beslut kring utvecklingsmiljöer samt säkerställa en sammanhängande och detaljerad design.
Arkitekt	Ansvarade för att utforma systemets övergripande arkitektur genom att definiera komponenter, gränssnitt och struktur, samt dokumentera dessa.
Testledare	Ansvarade för verifiering och validering av systemet, inklusive framtagning av testplan samt genomförande och dokumentation av tester.
Dokumentansvarig	Säkerställde att dokumentation följde givna mallar och riktlinjer samt att leveranser genomfördes enligt fastställda deadlines.
Kvalitetssamordnare	Ansvarade för att upprätthålla kvalitet i projektets arbete samt stödja gruppen genom planering, uppföljning och samordning av kvalitetsrelaterade aktiviteter.

4.3 Utveckling och systemarkitektur

Detta delkapitel beskriver hur Projekt Visir implementerades utifrån den arkitektur och arbetsfördelning som etablerats tidigare i projektet. Fokus ligger på hur *frontend*, *backend*, *datalösning* och externa integrationer konkret användes tillsammans för att realisera systemets centrala funktioner.

4.3.1 Systemarkitektur och design

Projekt Visir implementerades som en sammanhållen Next.js-applikation där *frontend* och *backend* utvecklades inom samma system. *Frontend* byggdes med React-komponenter och

backend med *API-routes*, vilket gav en tydlig struktur för hur användarinteraktion, serverlogik och datakommunikation kopplades samman. Applikationen driftsattes på Vercel, vilket gjorde det möjligt att samla hela lösningen i en gemensam driftmiljö.

I implementationen användes arkitekturen för att tydligt separera ansvar mellan klient och server. *Frontend* ansvarade för användargränssnitt och interaktion, medan *backend* ansvarade för databehandling, lönsamhetsrelaterad logik och kommunikation med externa system. *Frontend* kommunicerade med *backend* via *REST-API* och iLog-data hämtades endast av *backend*. Denna uppdelning gjorde det möjligt att samla integrations- och säkerhetskritisk funktionalitet i ett kontrollerat lager och samtidigt hålla klienten fokuserad på presentation och användarflöden.

Systemarkitekturen utformades även modulärt. Genom att dela upp systemet i separata delar för *frontend*, *backend*, databas och integrationer skapades en struktur där olika delar kan utvecklas och underhållas utan att hela systemet behöver omarbetas. Detta ligger i linje med forskning som visar att arkitektoniska kvaliteter och problem påverkar systemets underhållbarhet, särskilt vad gäller testbarhet och möjligheten att vidareutveckla systemets struktur [4].

4.3.2 Frontend-utveckling: React-komponenter och användargränssnitt

Frontendutvecklingen organiserades kring React-komponenter som tillsammans byggde upp systemets centrala delar av användargränssnittet. Detta omfattade bland annat komponenter för översiktsvyer, filtrering, detaljvisning och presentation av resultat. En komponentbaserad struktur gjorde det möjligt att arbeta modulärt med olika delar av gränssnittet och att stegvis koppla dessa till *backendens* dataflöden.

För att hantera information i gränssnittet användes *state management*. Detta var nödvändigt för att hålla reda på exempelvis valda filter, aktiv vy, hämtade resultat och tillfälliga värden i simuleringar. *State management* användes därmed som ett stöd för att synkronisera gränssnittets olika delar och säkerställa att användaren fick korrekt återkoppling vid interaktion med systemet.

Lönsamhetsvisualiseringarna implementerades med ett trafikljus-system där resultat kunde presenteras på ett snabbt och lättolkat sätt. Denna visualisering kompletterades av mer detaljerad information i anslutande vyer, så att användaren både kunde få en snabb överblick och vid behov fördjupa sig i bakomliggande data. *Frontenden* utformades även utifrån tydliga användarflöden, där användaren efter inloggning skulle kunna navigera mellan transportöversikter, filtrerade urval, detaljvyer och simuleringar. För layout och styling användes Tailwind CSS, vilket underlättade arbetet med *responsive design* och ett konsekvent gränssnitt.

4.3.3 Backend-utveckling: Next.js API-routes och serverlogik

Backendutvecklingen byggdes med Next.js *API-routes* och utgjorde systemets centrala logiska lager. *API-routes* implementerades för att hantera inkommande anrop från *frontend* och för att utföra de operationer som krävs för prisestimering, lönsamhetskalkyl, simulering och integration med iLog. Prisestimeringsdelen i *backend* utformades med stöd i tidigare arbete kring förbättrade prissättningsmodeller för lastbilstransporter [8]. Genom att samla denna funktionalitet i *backend* kunde affärslogik, säkerhet och databehandling hanteras separat från *frontend*.

I implementationen användes separata *API-routes* för olika typer av funktionalitet. Vissa

routes användes för att hämta eller förmedla data till *frontend*, medan andra användes för att behandla underlag för prisestimering och lönsamhetsbedömning. *Backend* innehöll även logik för simulering, där alternativa scenarier kunde behandlas utan att påverka externa produktionssystem. Detta gjorde det möjligt att skilja mellan ordinarie dataflöden och temporära beräkningar kopplade till användarens analysarbete.

Backend ansvarade också för all kommunikation med iLog och Supabase. Därmed samlades både integrationslogik och datakommunikation i samma lager, vilket förenklade kontrollen över hur information hämtas, bearbetas och görs tillgänglig för resten av systemet. Denna struktur gjorde *backend* till navet mellan *frontend*, databasen och de externa system som projektet är beroende av.

Utöver detta utvecklades *backends* serverlogik modulärt, så att olika delar som integrationslogik, beräkningslogik och datakommunikation kan utvecklas och underhållas var för sig. En sådan uppdelning är också förenlig med forskning som kopplar arkitektonisk struktur till underhållbarhetsrelaterade kvaliteter i mjukvarusystem [4].

4.3.4 Databaskonstruktion: PostgreSQL och Supabase

Databaskonstruktionen byggdes med PostgreSQL samt Supabase och användes som stöd för systemets analys-, integrations- och presentationsflöden. Databasen utformades för att kunna lagra transportrelaterad information, historiska underlag, beräknade resultat och *cache*-lagrad data från externa system.

I implementationen användes databasschemat för att strukturera tabeller för transportdata och lönsamhetsresultat. Dessa tabeller fungerade som underlag för *backends* beräkningar och för den information som sedan presenterades i *frontend*. Datamodellen förfinades successivt under projektets gång för att bättre stödja de datarelationer som behövdes i systemets centrala funktioner.

Databasen användes även för *caching* av iLog-data. Genom att mellanlagra hämtad information i Supabase kunde systemet återanvända data utan att varje användarinteraktion behövde leda till nya externa anrop. I implementationen blev Supabase därmed en gemensam datalösning för både *frontend* och *backend*: *backend* ansvarade för huvuddelen av lagring, hämtning och bearbetning, medan *frontend* använde den tillgängliga datan som grund för visning, uppdatering och användarinteraktion.

4.3.5 iLog API-integration

iLog-integrationen implementerades som en anslutning till ett *read-only API*. Systemet utformades därmed för att endast läsa information från iLog och inte skriva tillbaka några förändringar. Detta gjorde det möjligt att använda iLog som extern informationskälla utan att påverka kundens befintliga produktionsmiljö.

I implementationen var det endast *backend* som hade tillåtelse att hämta data från iLog. *Frontend* kommunicerade i stället med *backends API-routes*, som ansvarade för att hämta, validera och vidarebearbeta relevant information. Den data som hämtades från iLog cachades lokalt i Supabase för att förbättra prestanda och minska beroendet av upprepade externa anrop. På så sätt kunde systemet kombinera aktuell extern information med snabb lokal åtkomst i de delar av gränssnittet där användaren behövde direkt återkoppling.

4.3.6 Systemanatomi

Systemanatomin presenterar systemets komponenter och dataflödet mellan *frontend*, *backend*, Supabase och iLog. I rapporten används den för att ge en mer konkret bild av hur systemets delar samverkar under faktisk användning.

Systemanatomin synliggör hur användarens interaktion i *frontend* leder till anrop mot *backend*, hur *backend* hämtar eller lagrar information i Supabase, samt hur extern data från iLog förs in i systemet och görs tillgänglig för vidare bearbetning. Diagrammet fungerar därmed som en sammanfattning av ansvarsfördelningen mellan lagren och av de viktigaste beroendena i implementationen. Det användes också som stöd för att förstå i vilken ordning olika delar behövde kopplas samman under utvecklingsarbetet.

4.4 Testning och kvalitetssäkring

Detta delkapitel presenterar hur systemet testades för att säkerställa att det uppfyllde kundens krav och fungerade korrekt i sin avsedda miljö. För att garantera en stabil och robust produkt utvärderades systemet på flera olika testnivåer, från isolerade tekniska enhetstester till mer explorativa användbarhetstester. Testorganisationen byggde på ett delat ansvar inom projektgruppen. I praktiken innebar detta att varje enskild utvecklare ansvarade för att löpande testa sin egen kod. Samtidigt hanterades den mer övergripande testningen gemensamt för att säkerställa en hög kvalitet på den slutgiltiga produkten.

4.4.1 Enhetstestning

Enhetstester användes för att verifiera enskilda funktioner och moduler i systemet. Eftersom så pass ingående tester krävde god förståelse av det som testades, genomfördes testerna kontinuerligt av respektive utvecklare i samband med att ny kod implementerades. Utformningen och genomförandet av de specifika enhetstesterna överläts till den enskilde utvecklaren, vilket förutsatte ett stort personligt ansvar för den egna kodens kvalitet.

4.4.2 Integrationstestning

Integrationstester genomfördes för att säkerställa att systemets olika komponenter fungerade korrekt tillsammans, det vill säga kommunikationen mellan *frontend*, *backend*, Supabase och externa system. Dessa tester genomfördes först när tillräcklig funktionalitet fanns på plats, och därefter regelbundet eller vid större förändringar för att säkerställa att ny implementation hade skett korrekt. I projektet testades bland annat kommunikationen mellan *frontend*, *backend* och Supabase, samt integrationen med kundens API för data. Testerna kontrollerade också att kommunikationen använde rätt format.

4.4.3 System- och användbarhetstestning

Systemtester genomfördes för att kontrollera att systemet som helhet fungerade enligt ställda krav ur ett användarperspektiv. Detta inkluderade tester av grundläggande flöden såsom inloggning, inmatning, beräkningar, presentation av resultat och felmeddelanden. Det säkerställdes även att systemet fungerade oberoende av webbläsare.

Utöver de rent funktionella systemtesterna utfördes även mer explorativa användbarhetstester för att utvärdera tydligheten för slutanvändaren. Dessa tester fokuserade mer specifikt på att undersöka om en ny testperson kunde förstå och navigera i systemet utan instruktioner, samt hur väl systemets resultat och felmeddelanden kunde tolkas.

Båda dessa testkategorier genomfördes delvis tillsammans med kunden för att säkerställa att produkten uppfyllde deras behov och levde upp till deras förväntningar.

4.4.4 Bugfix och iterativ förbättring

Hantering av identifierade fel och buggar skedde iterativt under utvecklingens gång för att upprätthålla en hög kvalitet på koden. Om ett fel upptäcktes åtgärdades det i första hand direkt av den ansvariga utvecklaren, vilket var den primära metoden vid enhetstestning. Vid mer komplexa fel, eller problem som påverkade flera delar av systemet, dokumenterades felet och en plan för att åtgärda det bestämdes under nästkommande möte. För att säkerställa att befintlig funktionalitet inte påverkades negativt av nya ändringar, kördes berörda tester alltid om vid större uppdateringar.

4.4.5 Hantering av bias vid testning

Ett metodologiskt problem som uppstår i agila projekt, där utvecklare ofta ansvarar för att testa sin egen kod, är risken för så kallad bekräftelsebias (*confirmation bias*). Forskning inom Software Engineering visar att utvecklare och testare omedvetet tenderar att utforma testfall som enbart bekräftar att koden fungerar enligt det förväntade flödet snarare än att aktivt försöka identifiera oväntade fel och kantfall [9]. Denna studie slår även fast att bekräftelsebias förstärks signifikant när testning och utveckling sker under tidspress.

Eftersom projektet genomfördes under en fast tidsram, och initialt inom ett relativt obekant område, fanns det en risk att utvecklarna omedvetet enbart utformade tester som bekräftade det förväntade utfallet. För att motverka detta och garantera en hög kvalitet på slutprodukten förlitade sig kvalitetssäkringen aldrig uteslutande på tester skrivna av kodens skapare. Risken hanterades istället genom att den enskilda testningen kontinuerligt kompletterades med strukturerade integrationstester där andra gruppmedlemmar granskade koden, samt genom explorativa system- och användbarhetstester tillsammans med beställaren för att fånga upp oväntade scenarier ur ett verkligt slutanvändarperspektiv.



5 Resultat

5.1 Systembeskrivning

- TODO: Översiktlig beskrivning av system och dokument.
- TODO: Vad har kunden för värde hos det som skapats?
- TODO: Utvärdering av produkten.

5.2 Processbeskrivning

- TODO: Hur har ni arbetat i projektet?
- TODO: Dokumentera process i fokus, förbättra process, utvärdera förbättringen.
- TODO: Beskrivning av mätningar – inkl. mätningar av kvalitetskrav.

5.3 Gemensamma erfarenheter

- TODO: Överförbara erfarenheter.
- TODO: Goda och mindre bra erfarenheter.
- TODO: Erfarenheter från projektets alla faser.
- TODO: Tekniska och processrelaterade erfarenheter.



6 Diskussion

6.1 Resultat

- TODO: Fanns alternativa implementationssätt?
- TODO: Fanns det alternativa sätt att utveckla på?
- TODO: Vad återstår för att kunden ska få ut fullt värde av produkten?
- TODO: Lyckades ni förbättra/fortsätta något från tidigare projekt?
- TODO: Viktigaste lärdomar inför framtiden.

6.2 Metod

- TODO: Vilka konsekvenser fick de valda metoderna för resultaten?
- TODO: Fanns det alternativ?
- TODO: Källkritik.

6.3 Arbetet i ett vidare sammanhang

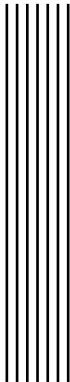
- TODO: Hållbarhetsaspekter – alla dimensioner behandlade, minst en strukturell effekt.
- TODO: Etiska aspekter.



7

Slutsats

- TODO: Sammanfatta syftet och frågeställningarna. I vilken utsträckning har målet uppnåtts, och vad är svaren på frågeställningarna?
- TODO: Beskriv konsekvenserna för målgruppen.



Referenser

- [1] Börjes Koncernen. *Börjes Koncernen - Hållbara transport- och logistiklösningar*. Accessed: 2026-03-07. 2026. URL: <https://borjeskoncernen.se/>.
- [2] CE Sweden. *An Overview of the Swedish Market for Third-Party Logistics (3PL) and Fulfillment Services*. Accessed: 2026-03-07. 2025. URL: <https://www.ce.se/an-overview-of-the-swedish-market-for-third-party-logistics-3pl-and-fulfillment-services/>.
- [3] D. Fjeld, M. Rönnqvist och P. Flisberg. "Developing methods for truck route selection and cost modeling in steeper topographies". I: *Forest Science* 66.4 (2020), s. 457–468. DOI: 10.1093/forsci/fxz019.
- [4] R. Jolak, S. Karlsson och F. Dobslaw. "An empirical investigation of the impact of architectural smells and refactoring on maintainability quality attributes". I: *Journal of Systems and Software* 223 (2025). DOI: 10.1016/j.jss.2025.112382.
- [5] Fizza Kamal. "Literature Survey on KANBAN: Opportunities and Challenges". I: *International Journal of Scientific and Research Publications* 10.11 (nov. 2020), s. 935–940. DOI: 10.29322/IJSRP.10.11.2020.p10786. URL: <http://dx.doi.org/10.29322/IJSRP.10.11.2020.p10786>.
- [6] PostgreSQL Global Development Group. *PostgreSQL 18 Documentation*. Accessed: 2026-03-25. PostgreSQL Global Development Group. 2025. URL: <https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf>.
- [7] React. *React Dev*. Accessed: 2026-03-21. 2026. URL: <https://react.dev/reference/react>.
- [8] M. Rönnqvist, P. Flisberg, G. Svenson och D. Noreland. "An enhanced pricing model for truck transportation: a case study in Swedish forestry". I: *International Transactions in Operational Research* 33.2 (2026), s. 775–797. DOI: 10.1111/itor.70062.
- [9] Iflaah Salman, Burak Turhan, Robert Ramač och Vladimir Mandić. "Confirmation Bias and Time Pressure: A Family of Experiments in Software Testing". I: *IEEE Transactions on Software Engineering* 49.12 (2023), s. 5203–5222. DOI: 10.1109/TSE.2023.3330400.
- [10] Sanket Vilas Salunke och Abdelkader Ouda. "A Performance Benchmark for the PostgreSQL and MySQL Databases". I: *Future Internet* 16.10 (2024). ISSN: 1999-5903. DOI: 10.3390/fi16100382. URL: <https://www.mdpi.com/1999-5903/16/10/382>.

-
- [11] Supabase Inc. *Data REST API*. Accessed: 2026-03-24. n.d. URL: <https://supabase.com/docs/guides/api>.
 - [12] Supabase Inc. *Supabase Database Guide*. Accessed: 2026-03-24. n.d. URL: <https://supabase.com/docs/guides/database/overview>.
 - [13] Supabase Inc. *Supabase Documentation*. Accessed: 2026-03-24. n.d. URL: <https://supabase.com/docs>.
 - [14] Trace Data Research. *Sweden Logistics and Warehousing Market Outlook to 2029*. Accessed: 2026-03-07. 2025. URL: <https://www.tracedataresearch.com/industry-report/sweden-logistics-and-warehousing-market>.
 - [15] Vercel, Inc. *Deploying to Vercel*. Accessed: 2026-03-24. n.d. URL: <https://vercel.com/docs/deployments>.
 - [16] Vercel, Inc. *Next.js App Router Documentation*. Accessed: 2026-03-24. n.d. URL: <https://nextjs.org/docs/app>.
 - [17] Vercel, Inc. *Next.js Pages Router Documentation*. Accessed: 2026-03-24. n.d. URL: <https://nextjs.org/docs/pages>.
 - [18] Vercel, Inc. *Vercel Documentation Overview*. Accessed: 2026-03-24. n.d. URL: <https://vercel.com/docs>.
 - [19] Vercel, Inc. *Vercel Functions Documentation*. Accessed: 2026-03-24. n.d. URL: <https://vercel.com/docs/functions>.
 - [20] Östersunds kommun. *Projektprocessen - Förstudie*. 2023. URL: <https://tekniskhandbok.ostersund.se/projektprocessen/2-2-forstudie/> (hämtad 2026-03-25).